МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Институт компьютерных наук
Кафедра прикладной математики

КУРСОВАЯ РАБОТА

по дисциплине: «Современные платформы разработки программного обеспечения»

на тему: «Разработка бухгалтерского приложения на C# и WPF»

Студент _____

группа _____

Руководитель _____

Липецк 2026 г.

Содержание

1. Техническое задание

1.1. Введение

Наименование программы: *BuhUchet*.
Тип программы: настольное бухгалтерское приложение.
Платформа: Windows, .NET 8, WPF.

Программа предназначена для ведения журнала хозяйственных операций, работы со справочником контрагентов, просмотра плана счетов и формирования оборотно-сальдовой ведомости (ОСВ).

1.2. Основание для разработки

Разработка выполнена в рамках учебного проекта по дисциплине «Современные платформы разработки программного обеспечения».

1.3. Назначение разработки

Основное назначение:

- регистрация и редактирование бухгалтерских проводок;
- хранение справочников (контрагенты, план счетов);
- построение отчета ОСВ за заданный период;
- экспорт и импорт отчетных данных в формате JSON.

1.4. Требования к программе

Средства разработки:

- язык программирования: C#;
- UI-фреймворк: WPF;
- целевая платформа: `net8.0-windows`;
- библиотека сериализации: Newtonsoft.Json.

Функциональные модули:

- модуль журнала операций;
- модуль контрагентов;
- модуль плана счетов;
- модуль отчетности (ОСВ);
- модуль автосохранения и хранения данных;
- модуль автодополнения полей ввода.

1.5. Функциональные требования

- добавление, изменение и удаление записей в журнале;
- фильтрация журнала по периоду и строке поиска;
- хранение ИНН и банковских реквизитов контрагентов;
- расчет входящих, оборотных и исходящих показателей ОСВ;
- экспорт ОСВ в JSON и загрузка ранее сохраненного отчета;
- автоматическое сохранение рабочих данных в `data.json`.

1.6. Нефункциональные требования

- удобный графический интерфейс;
- устойчивость к отсутствию или повреждению файла данных;
- минимальное время отклика при работе с таблицами;
- корректная работа с русской локалью при вводе сумм.

1.7. Требования к данным

Основной файл данных: `data.json`.

Структура данных:

- `Journal` — список проводок;

- `Counterparties` — список контрагентов;
- `Accounts` — план счетов;
- `SavedAt` — дата и время последнего сохранения.

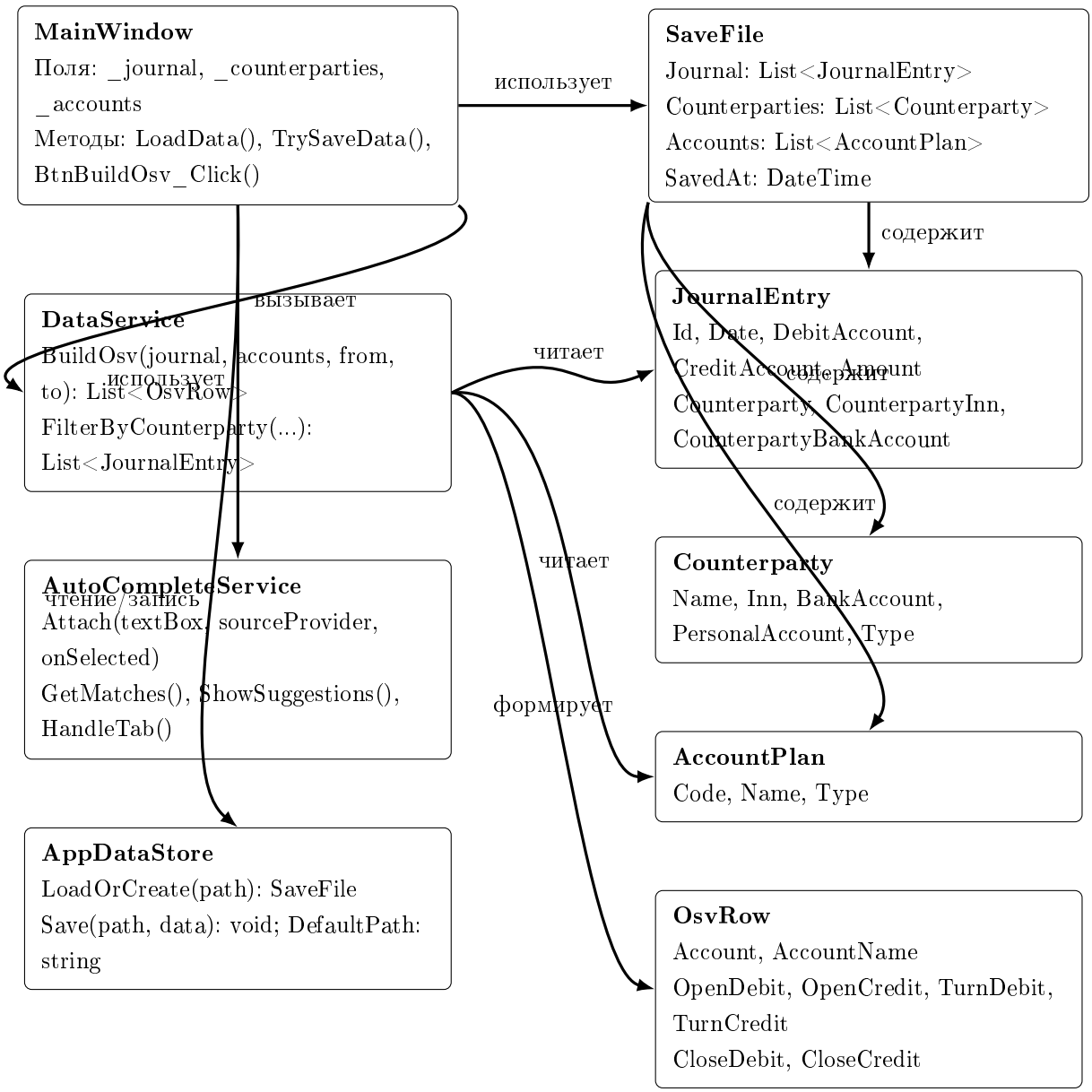
1.8. Этапы разработки

- проектирование модели данных;
- реализация интерфейса WPF;
- реализация бизнес-логики (журнал, контрагенты, ОСВ);
- добавление автодополнения и автосохранения;
- тестирование основных пользовательских сценариев.

2. Описание структуры приложения

2.1. Диаграмма классов

Ниже приведена UML-подобная диаграмма ключевых классов приложения.



2.2. Описание классов

2.2.1. Назначение классов предметной области

Предметная область приложения представлена сущностями:

- `JournalEntry` — бухгалтерская проводка (журнальная запись);
- `Counterparty` — контрагент (справочник контрагентов);
- `AccountPlan` — элемент плана счетов;
- `OsvRow` — строка оборотно-сальдовой ведомости;
- `SaveFile` — корневая структура сохранения данных.

2.2.2. Назначение сервисных классов

Сервисные компоненты:

- `AppDataStore` — чтение и запись файла данных `data.json`;
- `DataService` — расчёт ОСВ за период и фильтрация операций;
- `AutoCompleteService` — автодополнение (Tab и всплывающий список) в полях ввода.

2.3. Таблица описания классов

Основные файлы:

- `MainWindow.xaml` — разметка главного окна;
- `MainWindow.xaml.cs` — обработчики UI и сценарии работы;
- `Models.cs` — классы предметной области;
- `DataService.cs` — сервис расчетов отчетов;
- `AutoCompleteService.cs` — автодополнение в полях ввода;
- `AppDataStore.cs` — загрузка и сохранение данных.

2.4. Описание ключевых классов

Класс	**Назначение**	**Ключевые элементы**
`MainWindow`	Главный контроллер интерфейса	Навигация по разделам, обработчики кнопок, редактирование записей, экспорт/импорт ОСВ
`JournalEntry`	Модель проводки	Id, Date, DebitAccount, CreditAccount, Counterparty, Amount, Description
`Counterparty`	Модель контрагента	Name, Inn, BankAccount, PersonalAccount, Type

`AccountPlan`	Модель счета учета	Code, Name, Type
`OsvRow`	Строка оборотно-сальдовой ведомости	OpenDebit/OpenCredit, TurnDebit/TurnCredit, CloseDebit/CloseCredit
`DataService`	Формирование отчетов	`BuildOsv()`, `FilterByCounterparty()`
`AutoCompleteService`	Подсказки при вводе	`Attach()`, обработка Tab/Escape/Down, popup-список
`AppDataStore`	Работа с файлом данных	`LoadOrCreate()`, `Save()`, путь `DefaultPath`

2.5. Взаимодействие компонентов

1. При старте `MainWindow` загружает данные через `AppDataStore`.
2. Данные привязываются к `DataGrid` в виде `ObservableCollection`.
3. Пользователь изменяет записи; форма валидирует поля.
4. После изменений запускается отложенное автосохранение.
5. По запросу строится ОСВ через `DataService`.
6. Отчет может быть выгружен в JSON и загружен обратно.

2.6. Архитектурные особенности

- разделение на слой UI, модели и сервисы;
- применение `INotifyPropertyChanged` для реактивного обновления интерфейса;
- использование `DispatcherTimer` для debounce-автосохранения;
- работа с постоянным хранилищем через JSON-сериализацию.

3. Руководство оператора

3.1. Общие сведения о программе

Приложение содержит 4 раздела:

- Журнал операций;
- Контрагенты;
- План счетов;
- Отчеты (ОСВ).

3.2. Описание элементов интерфейса

Левая панель навигации содержит кнопки разделов и отображение статуса файла данных (`data.json`).
Журнал операций включает:

- таблицу операций (DataGrid) с колонками: №, Дата, Дт, Кт, Контрагент, Сумма, Содержание;
- панель фильтрации по периоду (DatePicker «с/по») и строку поиска;
- форму редактирования/добавления записи с валидацией обязательных полей;
- быстрые подсказки в полях (автодополнение по Tab и выпадающий список).

Контрагенты включают таблицу справочника и форму редактирования реквизитов (ИНН, банковский счёт, лицевой счёт, тип).
План счетов отображается в виде таблицы (код, наименование, тип).
Отчёты (ОСВ) включают выбор периода, таблицу ОСВ, строку итогов и кнопки экспорта/импорта JSON.

3.3. Необходимое программное обеспечение

- ОС: Windows 10/11;

- .NET: целевая платформа `net8.0-windows`;
- IDE для разработки/запуска: Visual Studio 2022+ (Desktop development with C#);
- (опционально) LaTeX-дистрибутив для сборки отчёта: TeX Live / MiKTeX.

3.4. Необходимое аппаратное обеспечение

Параметр	**Минимальные требования**	**Рекомендуемые требования**
Процессор	2 ядра, 2.0 ГГц	4 ядра, 2.5 ГГц+
ОЗУ	4 ГБ	8 ГБ+
Накопитель	300 МБ свободно	1 ГБ свободно
Экран	1366x768	1920x1080

3.5. Выполнение программы

1. Открыть проект `BuhUchet.sln` в Visual Studio.
2. Выполнить сборку и запуск (`F5`).
3. После запуска перейти в нужный раздел через левое меню.

3.6. Входные и выходные данные

Входные данные:

- данные проводок (дата, счета, контрагент, сумма, описание);
- реквизиты контрагентов (ИНН, счета, тип);
- период отчёта ОСВ (даты «с/по»);
- JSON-файл отчёта ОСВ при импорте.

Выходные данные:

- содержимое таблиц в UI;
- сформированная ОСВ (таблица и итоги);

- файл `data.json` (автосохранение);
- экспортированный JSON-файл отчёта ОСВ.

3.7. Сообщения оператору

Приложение использует строку статуса внизу окна и короткие сообщения:

- информационные: «[OK] Операция сохранена», «[OK] Контрагент сохранён», «[OK] ОСВ сформирована: N счетов», «[OK] Экспортировано: имя_файла»;
- предупреждения/ошибки: «[WARN] Укажите период», «[WARN] Введите корректную сумму», «[WARN] Ошибка экспорта: ...», «[WARN] Ошибка загрузки отчёта: ...», «[WARN] Ошибка автосохранения: ...».

3.8. Примеры использования

Пример 1: добавление проводки.

1. Раздел «Журнал операций» → кнопка «Добавить».
2. Заполнить: Дата, Дт, Кт, Контрагент, Сумма, Содержание.
3. Нажать «Сохранить». Внизу появится сообщение «[OK] Операция сохранена».

Пример 2: формирование ОСВ и экспорт.

1. Раздел «Отчёты (ОСВ)» → выбрать период.
2. Нажать «Сформировать». Проверить итоги.
3. Нажать «Экспорт ОСВ (JSON)» и сохранить файл.

3.9. Работа с журналом операций

1. Нажать «Добавить».
2. Заполнить дату, счета Дт/Кт, контрагента, сумму и описание.

3. Нажать «Сохранить».
4. Для изменения выбрать запись в таблице, отредактировать и сохранить.
5. Для удаления выбранной записи нажать кнопку удаления и подтвердить действие.

3.10. Работа с контрагентами

1. Перейти в раздел «Контрагенты».
2. Добавить или выбрать существующего контрагента.
3. Заполнить реквизиты (ИНН, банковский счет, лицевой счет, тип).
4. Сохранить изменения.

3.11. Формирование ОСВ

1. Перейти в раздел «Отчеты (ОСВ)».
2. Указать период «с» и «по».
3. Нажать «Сформировать».
4. При необходимости выполнить экспорт отчета в JSON.

4. Описание контрольного примера

4.1. Назначение

Контрольный пример демонстрирует корректность:

- ввода и хранения проводок;
- расчета итогов по счетам;
- формирования ОСВ за выбранный период.

4.2. Исходные данные

Период: 01.01.2026–31.01.2026.
Проводки:

1. Дт 10 Кт 60, сумма 10000;
2. Дт 20 Кт 10, сумма 4000;
3. Дт 62 Кт 90, сумма 15000;
4. Дт 51 Кт 62, сумма 7000.

4.3. Дополнение для примера

В файле `data.json` данные хранятся в виде объекта `SaveFile`. Для проверки также допустимо подготовить тестовый `data.json` вручную (например, добавить несколько счетов в `Accounts` и несколько операций в `Journal`).

4.4. Сценарий для проверки

1. Внести проводки в журнал.
2. Сформировать ОСВ за январь 2026.
3. Проверить обороты по счетам 10, 20, 51, 60, 62, 90.
4. Проверить корректность итоговых строк.

4.5. Визуализация

Результатом выполнения сценария является заполненная таблица ОСВ в разделе «Отчёты (ОСВ)» и строка итогов. Таблица включает колонки начального сальдо, оборотов и конечного сальдо.

4.6. Результаты расчёта

Ожидаемые обороты за период:

- счёт 10: оборот Дт = 10000, оборот Кт = 4000;
- счёт 20: оборот Дт = 4000;

- счёт 60: оборот Кт = 10000;
- счёт 62: оборот Дт = 15000, оборот Кт = 7000;
- счёт 90: оборот Кт = 15000;
- счёт 51: оборот Дт = 7000.

Конечные сальдо рассчитываются формулами, используемыми в `OsvRow`:

$$\text{CloseDebit} = \max(0,\ OpenDebit - OpenCredit + TurnDebit - TurnCredit) \tag{1}$$

$$\text{CloseCredit} = \max(0,\ OpenCredit - OpenDebit + TurnCredit - TurnDebit) \tag{2}$$

4.7. Проверка программы

4.7.1. Состав технических средств

ПК под управлением Windows 10/11, установленный .NET 8 runtime (или Visual Studio для запуска из IDE).

4.7.2. Формирование исходных данных

Исходные данные формируются через пользовательский интерфейс (журнал операций), либо путём заполнения `data.json` и последующего запуска приложения.

4.7.3. Описание действий оператора

1. Запустить приложение.
2. Внести операции (см. раздел 4.2).
3. Перейти в «Отчёты (ОСВ)», указать период, сформировать ОСВ.
4. Сравнить обороты и итоги со значениями из раздела 4.3.

4.8. Пример выполнения

После ввода операций и формирования ОСВ оператор видит:

- список задействованных счетов;
- обороты по дебету/кредиту;
- рассчитанные конечные сальдо;
- строку итогов (суммы оборотов и конечных сальдо).

4.9. Проверка ошибок

Сценарий	**Ожидаемая реакция**	**Результат**
Пустая дата операции	Сообщение «Укажите дату операции»	Блокируется сохранение
Некорректная сумма (буквы/0/отрицательная)	Сообщение «Введите корректную сумму»	Блокируется сохранение
Экспорт в недоступную папку	«Ошибка экспорта: ...»	Сообщение об ошибке
Импорт повреждённого JSON	«Ошибка загрузки отчёта: ...»	Сообщение об ошибке

- для каждого использованного счета отображаются обороты по дебету и кредиту;
- рассчитаны начальные и конечные сальдо;
- итоговые суммы по ОСВ отображаются в статусной области отчета;
- экспортированный JSON содержит разделы `Rows` и `Totals`.

5. Сообщения об ошибках

5.1. Классификация ошибок

- ошибки валидации пользовательского ввода;

- ошибки чтения/записи файла;
- ошибки парсинга формата отчета.

5.2. Таблица ошибок

Код	Ситуация	Сообщение	Действие
VAL-001	Не указана дата операции	«Укажите дату операции»	Блокировка сохранения
VAL-002	Некорректная сумма	«Введите корректную сумму»	Блокировка сохранения
IO-001	Ошибка экспорта отчета	«Ошибка экспорта: ...»	Показ статуса ошибки
IO-002	Ошибка загрузки отчета	«Ошибка загрузки отчёта: ...»	Показ статуса ошибки
IO-003	Ошибка автосохранения	«Ошибка автосохранения: ...»	Уведомление оператора

5.3. Примеры обработки ошибок в коде

Валидация полей при сохранении проводки (пример логики из `MainWindow.xaml.cs`):

```
if (PickDate.SelectedDate == null) { ShowStatus("
                              ", true); return; }
if (string.IsNullOrWhiteSpace(TxtDebit.Text)) { ShowStatus("
                                          ", true); return; }
if (string.IsNullOrWhiteSpace(TxtCredit.Text)) { ShowStatus("
                                            ", true); return; }
if (!TryParseAmount(TxtAmount.Text, out decimal amount) || amount
    <= 0)
{ ShowStatus("                                                  ",
   true); return; }
```

Обработка исключений при экспорте/импорте отчёта реализована через `try/catch` с выводом сообщения в строку статуса.

5.4. Визуальное отображение ошибок

Ошибки отображаются в нижней статусной строке:

- ошибки: префикс «[WARN]», цвет текста — *LightSalmon*;
- успешные операции: префикс «[OK]», цвет текста — *LightGreen*.

5.5. Логирование ошибок

В текущей версии приложения отдельный файл логов не ведётся: информация выводится в UI (строка статуса) и в исключительных ситуациях может быть доступна через стандартные средства отладки.
Рекомендуемое улучшение: добавление записи ошибок в файл (например, `error_log.txt`) в формате `[Дата] [Код] [Сообщение]` в обработчиках исключений автосохранения, экспорта и импорта.

5.6. Рекомендации для тестировщиков

- проверить валидацию обязательных полей (пустая дата, пустые счета, неверная сумма);
- проверить корректность расчёта ОСВ на небольшом наборе проводок с ручным подсчётом;
- проверить автосохранение: изменить данные и убедиться, что `data.json` обновляется;
- проверить импорт ОСВ из корректного JSON и из повреждённого JSON.

6. Список литературы

1. ГОСТ 19.401-78. ЕСПД. Текст программы. Требования к содержанию и оформлению.
2. ГОСТ 19.505-79. ЕСПД. Руководство оператора. Требования к содержанию и оформлению.
3. Microsoft Docs. WPF documentation. `https://learn.microsoft.com/dotnet/desktop/wpf/`

4. Microsoft Docs. C# documentation. https://learn.microsoft.com/dotnet/csharp/

5. Newtonsoft.Json documentation. https://www.newtonsoft.com/json

A. Приложение А. Текст программы

В приложении приведены ключевые файлы проекта бухгалтерского приложения *BuhUchet*.

A.1. BuhUchet.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseWPF>true</UseWPF>
    <AssemblyName>BuhUchet</AssemblyName>
    <RootNamespace>BuhUchet</RootNamespace>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.3"
       />
  </ItemGroup>
  <ItemGroup>
    <None Update="data.json">
      <CopyToOutputDirectory>PreserveNewest</
         CopyToOutputDirectory>
      <CopyToPublishDirectory>PreserveNewest</
         CopyToPublishDirectory>
    </None>
  </ItemGroup>
  <ItemGroup>
    <Content Include="Assets\appicon.png">
      <CopyToOutputDirectory>PreserveNewest</
         CopyToOutputDirectory>
      <CopyToPublishDirectory>PreserveNewest</
         CopyToPublishDirectory>
    </Content>
  </ItemGroup>
</Project>
```

A.2. MainWindow.xaml

```
<Window x:Class="BuhUchet.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/
           presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:BuhUchet"
        Title="
                                "
        Height="720" Width="1440"
        MinHeight="600" MinWidth="900"
        Background="#F0F2F5"
        Icon="pack://siteoforigin:,,,/Assets/appicon.png"
        WindowStartupLocation="CenterScreen">

    <Window.Resources>
        <!--                                                    -->
        <SolidColorBrush x:Key="PrimaryBrush"     Color="#1A3A5C"
           />
        <SolidColorBrush x:Key="AccentBrush"      Color="#2E7DB5"
           />
        <SolidColorBrush x:Key="SuccessBrush"     Color="#27AE60"
           />
        <SolidColorBrush x:Key="DangerBrush"      Color="#E74C3C"
           />
        <SolidColorBrush x:Key="WarnBrush"        Color="#F39C12"
           />
        <SolidColorBrush x:Key="SurfaceBrush"     Color="#FFFFFF"
           />
        <SolidColorBrush x:Key="BorderBrush2"     Color="#D0D7DE"
           />
        <SolidColorBrush x:Key="MutedBrush"       Color="#6C757D"
           />
        <SolidColorBrush x:Key="SidebarBrush"     Color="#1A3A5C"
           />

        <!--                                              (sidebar)
            -->
        <Style x:Key="NavButton" TargetType="Button">
            <Setter Property="Background"       Value="
               Transparent"/>
            <Setter Property="Foreground"       Value="#A8C7E8"/>
```

```
        <Setter Property="BorderThickness"  Value="0"/>
        <Setter Property="HorizontalContentAlignment" Value="
           Left"/>
        <Setter Property="Padding"          Value="16,10"/>
        <Setter Property="FontSize"         Value="13"/>
        <Setter Property="Cursor"           Value="Hand"/>
        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="Button">
                    <Border x:Name="bd" Background="{
                       TemplateBinding Background}"
                            CornerRadius="6" Margin="6,2">
                        <ContentPresenter HorizontalAlignment
                           ="{TemplateBinding
                           HorizontalContentAlignment}"
                                          VerticalAlignment="
                                             Center"
                                          Margin="{
                                             TemplateBinding
                                             Padding}"/>
                    </Border>
                    <ControlTemplate.Triggers>
                        <Trigger Property="IsMouseOver" Value
                           ="True">
                            <Setter TargetName="bd" Property=
                               "Background" Value="#2E4D6B"/>
                            <Setter Property="Foreground"
                               Value="White"/>
                        </Trigger>
                        <Trigger Property="IsPressed" Value="
                           True">
                            <Setter TargetName="bd" Property=
                               "Background" Value="#2E7DB5"/>
                        </Trigger>
                    </ControlTemplate.Triggers>
                </ControlTemplate>
            </Setter.Value>
        </Setter>
    </Style>

    <!--                                                    -->
```

```
<Style x:Key="NavButtonActive" TargetType="Button"
   BasedOn="{StaticResource NavButton}">
    <Setter Property="Background" Value="#2E7DB5"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="FontWeight" Value="SemiBold"/>
</Style>

<!--
                    -->
<Style x:Key="ActionButton" TargetType="Button">
    <Setter Property="Background"       Value="#2E7DB5"/>
    <Setter Property="Foreground"       Value="White"/>
    <Setter Property="BorderThickness"  Value="0"/>
    <Setter Property="Padding"          Value="14,7"/>
    <Setter Property="FontSize"         Value="12"/>
    <Setter Property="Cursor"           Value="Hand"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="Button">
                <Border x:Name="bd" Background="{
                   TemplateBinding Background}"
                        CornerRadius="5">
                    <ContentPresenter HorizontalAlignment
                       ="Center" VerticalAlignment="
                       Center"
                                     Margin="{
                                        TemplateBinding
                                        Padding}"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsMouseOver" Value
                       ="True">
                        <Setter TargetName="bd" Property=
                           "Background" Value="#1A6A9F"/>
                    </Trigger>
                    <Trigger Property="IsEnabled" Value="
                       False">
                        <Setter TargetName="bd" Property=
                           "Background" Value="#B0C4D8"/>
                    </Trigger>
                </ControlTemplate.Triggers>
```

```
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

<!--                                                    -->
<Style x:Key="DangerButton" TargetType="Button" BasedOn="
   {StaticResource ActionButton}">
    <Setter Property="Background" Value="#E74C3C"/>
    <Style.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
            <Setter Property="Background" Value="#C0392B"
               />
        </Trigger>
    </Style.Triggers>
</Style>

<!--            TextBox -->
<Style x:Key="FormTextBox" TargetType="TextBox">
    <Setter Property="BorderBrush"      Value="#D0D7DE"/>
    <Setter Property="BorderThickness"  Value="1"/>
    <Setter Property="Padding"          Value="8,6"/>
    <Setter Property="FontSize"         Value="12"/>
    <Setter Property="Background"       Value="White"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="TextBox">
                <Border x:Name="bd" Background="{
                   TemplateBinding Background}"
                        BorderBrush="{TemplateBinding
                           BorderBrush}"
                        BorderThickness="{TemplateBinding
                            BorderThickness}"
                        CornerRadius="4">
                    <ScrollViewer x:Name="
                       PART_ContentHost" Margin="0"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsFocused" Value="
                       True">
```

```
                                <Setter TargetName="bd" Property=
                                    "BorderBrush" Value="#2E7DB5"/
                                    >
                                <Setter TargetName="bd" Property=
                                    "BorderThickness" Value="1.5"/
                                    >
                            </Trigger>
                        </ControlTemplate.Triggers>
                    </ControlTemplate>
                </Setter.Value>
            </Setter>
        </Style>

        <!--            Label            -->
        <Style x:Key="FormLabel" TargetType="TextBlock">
            <Setter Property="FontSize"     Value="11"/>
            <Setter Property="FontWeight"   Value="SemiBold"/>
            <Setter Property="Foreground"   Value="#4A5568"/>
            <Setter Property="Margin"       Value="0,0,0,3"/>
        </Style>

        <!-- DataGrid              -->
        <Style TargetType="DataGridRow">
            <Style.Triggers>
                <Trigger Property="AlternationIndex" Value="1">
                    <Setter Property="Background" Value="#F8FAFC"
                        />
                </Trigger>
                <Trigger Property="IsSelected" Value="True">
                    <Setter Property="Background" Value="#D6EAF8"
                        />
                </Trigger>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#EBF5FB"
                        />
                </Trigger>
            </Style.Triggers>
        </Style>
    </Window.Resources>

    <Grid>
```

```
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="200"/>
    <ColumnDefinition Width="*"/>
</Grid.ColumnDefinitions>

<!--            SIDEBAR

    -->
<Border Grid.Column="0" Background="#1A3A5C">
    <DockPanel>
        <!--                    -->
        <Border DockPanel.Dock="Top" Padding="16,20,16,16
           "
                BorderBrush="#2E4D6B" BorderThickness="
                   0,0,0,1">
            <StackPanel>
                <TextBlock Text="                    "
                           Foreground="White" FontSize="
                              18"
                           FontWeight="Bold"/>
                <TextBlock Text="v1.0
                               "
                           Foreground="#6B9EC7" FontSize=
                              "10"
                           Margin="0,2,0,0"/>
            </StackPanel>
        </Border>

        <!--                      -->
        <Border Padding="6,4">
            <StackPanel>
                <TextBlock Text="              "
                   Foreground="#4A6D8C"
                           FontSize="10" FontWeight="Bold
                              "
                           Margin="10,0,0,6"/>
                <Button x:Name="BtnJournal"
                   Content="
                                   "   Style="{
                   StaticResource NavButtonActive}" Click
                   ="NavButton_Click" Tag="Journal"/>
```

```
                <Button x:Name="BtnCounterparties"
                   Content="                              "
                          Style="{StaticResource
                   NavButton}"       Click="
                   NavButton_Click" Tag="Counterparties"/
                   >
                <Button x:Name="BtnAccounts"
                   Content="                              "
                          Style="{StaticResource
                   NavButton}"       Click="
                   NavButton_Click" Tag="Accounts"/>
                <Button x:Name="BtnReports"
                   Content="                    (      )"
                         Style="{StaticResource NavButton
                   }"       Click="NavButton_Click" Tag="
                   Reports"/>
            </StackPanel>
        </Border>

        <!--                                        -->
        <Border DockPanel.Dock="Bottom" Padding="14,10"
                BorderBrush="#2E4D6B" BorderThickness="
                   0,1,0,0">
            <StackPanel>
                <TextBlock x:Name="TxtFileName" Text="
                   data.json"
                         Foreground="#6B9EC7" FontSize=
                            "10"
                         TextTrimming="
                            CharacterEllipsis"/>
                <TextBlock x:Name="TxtSavedAt" Text="
                                  "
                         Foreground="#4A6D8C" FontSize=
                            "10"/>
            </StackPanel>
        </Border>
    </DockPanel>
</Border>

<!--            MAIN CONTENT
```

```
        -->
    <Grid Grid.Column="1">
        <Grid.RowDefinitions>
            <RowDefinition Height="*"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!--

            -->
        <Grid x:Name="PanelJournal" Grid.Row="0">
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="*"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>

            <!--                    +            -->
            <Border Grid.Row="0" Background="White" Padding="
               20,14"
                    BorderBrush="#E2E8F0" BorderThickness="
                       0,0,0,1">
                <Grid>
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="Auto"/>
                    </Grid.ColumnDefinitions>
                    <StackPanel>
                        <TextBlock Text="

                                        "
                                    FontSize="18" FontWeight="
                                       SemiBold" Foreground="
                                       #1A3A5C"/>
                        <TextBlock x:Name="TxtJournalCount"
                           Text="0                "
                                    FontSize="12" Foreground="
                                       #6C757D" Margin="
                                       0,2,0,0"/>
                    </StackPanel>
```

```
<StackPanel Grid.Column="1" Orientation="
   Horizontal" VerticalAlignment="Center"
   >
    <TextBlock Text="  "
       VerticalAlignment="Center" Margin=
       "0,0,6,0"
                FontSize="11" Foreground="
                   #6C757D"/>
    <DatePicker x:Name="PickJournalFrom"
       Width="120" Margin="0,0,8,0"
                 FontSize="12"
                    SelectedDateChanged="
                    JournalFilter_Changed"
                    />
    <TextBlock Text="    "
       VerticalAlignment="Center" Margin=
       "0,0,6,0"
                FontSize="11" Foreground="
                   #6C757D"/>
    <DatePicker x:Name="PickJournalTo"
       Width="120" Margin="0,0,8,0"
                 FontSize="12"
                    SelectedDateChanged="
                    JournalFilter_Changed"
                    />
    <Button x:Name="BtnJournalReset"
       Content="          "
             Margin="0,0,10,0"
             Padding="10,7" FontSize="12"
             Click="BtnJournalReset_Click"
             Background="#EDF2F7"
                Foreground="#4A5568"
                BorderThickness="0"
             Cursor="Hand">
     <Button.Template>
             <ControlTemplate TargetType="
                Button">
                 <Border x:Name="bd"
                    Background="{
                    TemplateBinding
                    Background}"
```

```
                                CornerRadius="5">
                                  <ContentPresenter
                                     HorizontalAlignment
                                     ="Center"
                                     VerticalAlignment=
                                     "Center"
                                                    Margin
                                                       =
                                                       "
                                                       {
                                                       Templat

                                                       Padding
                                                       }
                                                       "
                                                       /
                                                       >

                            </Border>
                            <ControlTemplate.Triggers
                               >
                                  <Trigger Property="
                                     IsMouseOver" Value
                                     ="True">
                                       <Setter
                                          TargetName="bd
                                          " Property="
                                          Background"
                                          Value="#E2E8F0
                                          "/>
                                  </Trigger>
                            </ControlTemplate.
                               Triggers>
                        </ControlTemplate>
                    </Button.Template>
                </Button>
                <TextBox x:Name="TxtSearch" Width="
                   220" Margin="0,0,8,0"
                         Style="{StaticResource
                            FormTextBox}"
```

```
                                TextChanged="
                                   TxtSearch_TextChanged">
                            <TextBox.Resources>
                                <Style TargetType="Border">
                                    <Setter Property="
                                       CornerRadius" Value="4
                                       "/>
                                </Style>
                            </TextBox.Resources>
                        </TextBox>
                        <TextBlock Text="    "
                           VerticalAlignment="Center" Margin=
                           "-34,0,14,0"
                                   Foreground="#6C757D"
                                      FontSize="14"
                                      IsHitTestVisible="False
                                      "/>
                        <Button Content="+                 "
                           Style="{StaticResource
                           ActionButton}"
                                Click="BtnAddEntry_Click"/>
                    </StackPanel>
                </Grid>
            </Border>

            <!-- DataGrid -->
            <DataGrid x:Name="GridJournal" Grid.Row="1"
                      AutoGenerateColumns="False"
                      CanUserAddRows="False"
                      AlternationCount="2"
                      GridLinesVisibility="Horizontal"
                      HorizontalGridLinesBrush="#EDF2F7"
                      HeadersVisibility="Column"
                      SelectionMode="Single"
                      Background="White"
                      RowHeight="34"
                      FontSize="12"
                      BorderThickness="0"
                      Margin="0"
                      SelectionChanged="
                         GridJournal_SelectionChanged">
```

```
<DataGrid.ColumnHeaderStyle>
    <Style TargetType="DataGridColumnHeader">
        <Setter Property="Background"   Value
          ="#F8FAFC"/>
        <Setter Property="Foreground"   Value
          ="#4A5568"/>
        <Setter Property="FontWeight"   Value
          ="SemiBold"/>
        <Setter Property="FontSize"     Value
          ="11"/>
        <Setter Property="Padding"      Value
          ="10,8"/>
        <Setter Property="BorderBrush"  Value
          ="#E2E8F0"/>
        <Setter Property="BorderThickness"
          Value="0,0,1,1"/>
    </Style>
</DataGrid.ColumnHeaderStyle>
<DataGrid.Columns>
    <DataGridTextColumn Header="   "
                Binding="{Binding Id}"
                 Width="50"  IsReadOnly="
      True"/>
    <DataGridTextColumn Header="        "
              Binding="{Binding Date,
      StringFormat=dd.MM.yyyy}" Width="90"
      IsReadOnly="True"/>
    <DataGridTextColumn Header="    "
               Binding="{Binding
      DebitAccount}"  Width="60"  IsReadOnly
      ="True"/>
    <DataGridTextColumn Header="    "
               Binding="{Binding
      CreditAccount}" Width="60"  IsReadOnly
      ="True"/>
    <DataGridTextColumn Header="
                          "  Binding="{
      Binding Counterparty}"  Width="180"
      IsReadOnly="True"/>
    <DataGridTextColumn Header="          ,
             ." Binding="{Binding Amount,
```

```
                StringFormat=N2}" Width="110"
                IsReadOnly="True"/>
            <DataGridTextColumn Header="
                                        "  Binding="{
                Binding Description}"  Width="*"
                IsReadOnly="True"/>
        </DataGrid.Columns>
    </DataGrid>

    <!--
                    -->
    <Border x:Name="PanelEntryEdit" Grid.Row="2"
            Background="White" Padding="20,14"
            BorderBrush="#E2E8F0" BorderThickness="
                0,1,0,0"
            Visibility="Collapsed">
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="90"/>
                <ColumnDefinition Width="80"/>
                <ColumnDefinition Width="80"/>
                <ColumnDefinition Width="190"/>
                <ColumnDefinition Width="140"/>
                <ColumnDefinition Width="190"/>
                <ColumnDefinition Width="110"/>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="Auto"/>
            </Grid.ColumnDefinitions>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>

            <TextBlock Grid.Row="0" Grid.Column="0"
                Text="         " Style="{StaticResource
                 FormLabel}"/>
            <TextBlock Grid.Row="0" Grid.Column="1"
                Text="              " Style="{
                StaticResource FormLabel}"/>
            <TextBlock Grid.Row="0" Grid.Column="2"
                Text="              " Style="{
```

```
   StaticResource FormLabel}"/>
<TextBlock Grid.Row="0" Grid.Column="3"
   Text="                        (Tab
                                 )" Style="
   {StaticResource FormLabel}"/>
<TextBlock Grid.Row="0" Grid.Column="4"
   Text="      " Style="{StaticResource
   FormLabel}"/>
<TextBlock Grid.Row="0" Grid.Column="5"
   Text="        .         " Style="{
   StaticResource FormLabel}"/>
<TextBlock Grid.Row="0" Grid.Column="6"
   Text="          " Style="{
   StaticResource FormLabel}"/>
<TextBlock Grid.Row="0" Grid.Column="7"
   Text="                      " Style="{
   StaticResource FormLabel}"/>

<DatePicker x:Name="PickDate" Grid.Row="1
   " Grid.Column="0"
            Margin="0,0,8,0"
               VerticalContentAlignment="
               Center"
            FontSize="12"/>
<TextBox x:Name="TxtDebit"       Grid.Row
   ="1" Grid.Column="1" Margin="0,0,8,0"
   Style="{StaticResource FormTextBox}"
   ToolTip="                       (Tab
                                    )"/>
<TextBox x:Name="TxtCredit"      Grid.Row
   ="1" Grid.Column="2" Margin="0,0,8,0"
   Style="{StaticResource FormTextBox}"
   ToolTip="                         (Tab
                                    )"/>
<TextBox x:Name="TxtCounterparty" Grid.
   Row="1" Grid.Column="3" Margin="
   0,0,8,0" Style="{StaticResource
   FormTextBox}"/>
<TextBox x:Name="TxtCounterpartyInn" Grid
   .Row="1" Grid.Column="4" Margin="
   0,0,8,0" Style="{StaticResource
```

```
                FormTextBox}" IsReadOnly="True"
                Background="#F8FAFC"/>
            <TextBox x:Name="TxtCounterpartyBank"
                Grid.Row="1" Grid.Column="5" Margin="
                0,0,8,0" Style="{StaticResource
                FormTextBox}" IsReadOnly="True"
                Background="#F8FAFC"/>
            <TextBox x:Name="TxtAmount"      Grid.Row
                ="1" Grid.Column="6" Margin="0,0,8,0"
                Style="{StaticResource FormTextBox}"/>
            <TextBox x:Name="TxtDescription" Grid.Row
                ="1" Grid.Column="7" Margin="0,0,8,0"
                Style="{StaticResource FormTextBox}"/>

            <StackPanel Grid.Row="1" Grid.Column="8"
                Orientation="Horizontal">
                 <Button Content="
                                    " Style="{
                    StaticResource ActionButton}"
                         Margin="0,0,6,0" Click="
                            BtnSaveEntry_Click"/>
                 <Button Content="   " Style="{
                    StaticResource DangerButton}"
                         Width="40" Height="34"
                            Padding="0" FontSize="14"
                            Click="
                            BtnDeleteEntry_Click"
                         ToolTip="
                                        "/>
            </StackPanel>
        </Grid>
    </Border>
</Grid>

<!--

    -->
<Grid x:Name="PanelCounterparties" Grid.Row="0"
   Visibility="Collapsed">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
```

```
        <RowDefinition Height="*"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>

    <Border Grid.Row="0" Background="White" Padding="
       20,14"
            BorderBrush="#E2E8F0" BorderThickness="
               0,0,0,1">
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="Auto"/>
            </Grid.ColumnDefinitions>
            <TextBlock Text="
                                        "
                       FontSize="18" FontWeight="
                          SemiBold" Foreground="#1
                          A3A5C"/>
            <Button Grid.Column="1" Content="+

                                        "
                    Style="{StaticResource
                       ActionButton}" Click="
                       BtnAddCounterparty_Click"/>
        </Grid>
    </Border>

    <DataGrid x:Name="GridCounterparties" Grid.Row="1
       "
              AutoGenerateColumns="False"
                 CanUserAddRows="False"
              AlternationCount="2"
                 GridLinesVisibility="Horizontal"
              HorizontalGridLinesBrush="#EDF2F7"
                 HeadersVisibility="Column"
              SelectionMode="Single" Background="
                 White" RowHeight="34"
              FontSize="12" BorderThickness="0"
              SelectionChanged="
                 GridCounterparties_SelectionChanged"
                 >
```

```
        <DataGrid.ColumnHeaderStyle>
            <Style TargetType="DataGridColumnHeader">
                <Setter Property="Background"   Value
                  ="#F8FAFC"/>
                <Setter Property="Foreground"   Value
                  ="#4A5568"/>
                <Setter Property="FontWeight"   Value
                  ="SemiBold"/>
                <Setter Property="FontSize"     Value
                  ="11"/>
                <Setter Property="Padding"      Value
                  ="10,8"/>
                <Setter Property="BorderBrush"  Value
                  ="#E2E8F0"/>
                <Setter Property="BorderThickness"
                  Value="0,0,1,1"/>
            </Style>
        </DataGrid.ColumnHeaderStyle>
        <DataGrid.Columns>
            <DataGridTextColumn Header="
                                        " Binding="{
              Binding Name}" Width="*"   IsReadOnly=
              "True"/>
            <DataGridTextColumn Header="      "
                        Binding="{Binding Inn}"
              Width="130" IsReadOnly="True"/>
            <DataGridTextColumn Header="        .
                        "   Binding="{Binding
              BankAccount}" Width="160" IsReadOnly="
              True"/>
            <DataGridTextColumn Header="
                                        " Binding="{
              Binding PersonalAccount}" Width="130"
              IsReadOnly="True"/>
            <DataGridTextColumn Header="      "
                        Binding="{Binding Type}"
              Width="120" IsReadOnly="True"/>
        </DataGrid.Columns>
    </DataGrid>

    <!--                                        -->
```

```
<Border x:Name="PanelCpEdit" Grid.Row="2"
        Background="White" Padding="20,14"
        BorderBrush="#E2E8F0" BorderThickness="
          0,1,0,0"
        Visibility="Collapsed">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="130"/>
            <ColumnDefinition Width="160"/>
            <ColumnDefinition Width="130"/>
            <ColumnDefinition Width="130"/>
            <ColumnDefinition Width="Auto"/>
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Grid.Column="0"
          Text="                        " Style=
          "{StaticResource FormLabel}"/>
        <TextBlock Grid.Row="0" Grid.Column="1"
          Text="      " Style="{StaticResource
          FormLabel}"/>
        <TextBlock Grid.Row="0" Grid.Column="2"
          Text="        .         " Style="{
          StaticResource FormLabel}"/>
        <TextBlock Grid.Row="0" Grid.Column="3"
          Text="                        " Style="
          {StaticResource FormLabel}"/>
        <TextBlock Grid.Row="0" Grid.Column="4"
          Text="      " Style="{StaticResource
          FormLabel}"/>

        <TextBox x:Name="TxtCpName" Grid.Row="1"
          Grid.Column="0" Margin="0,0,8,0" Style
          ="{StaticResource FormTextBox}"/>
        <TextBox x:Name="TxtCpInn"  Grid.Row="1"
          Grid.Column="1" Margin="0,0,8,0" Style
          ="{StaticResource FormTextBox}"/>
```

```
                <TextBox x:Name="TxtCpBankAccount" Grid.
                   Row="1" Grid.Column="2" Margin="
                   0,0,8,0" Style="{StaticResource
                   FormTextBox}"/>
                <TextBox x:Name="TxtCpPersonalAccount"
                   Grid.Row="1" Grid.Column="3" Margin="
                   0,0,8,0" Style="{StaticResource
                   FormTextBox}"/>
                <ComboBox x:Name="CbCpType" Grid.Row="1"
                   Grid.Column="4" Margin="0,0,8,0"
                          FontSize="12" Padding="6,5"
                          VerticalContentAlignment="
                             Center">
                    <ComboBoxItem>                  </
                       ComboBoxItem>
                    <ComboBoxItem>                    </
                       ComboBoxItem>
                    <ComboBoxItem>            </
                       ComboBoxItem>
                </ComboBox>

                <StackPanel Grid.Row="1" Grid.Column="5"
                   Orientation="Horizontal">
                    <Button Content="
                                        " Style="{
                       StaticResource ActionButton}"
                             Margin="0,0,6,0" Click="
                                BtnSaveCp_Click"/>
                    <Button Content="   " Style="{
                       StaticResource DangerButton}"
                             Width="40" Height="34"
                                Padding="0" FontSize="14"
                             Click="BtnDeleteCp_Click"
                                ToolTip="              "/>
                </StackPanel>
            </Grid>
        </Border>
    </Grid>

    <!--
```

```
    -->
<Grid x:Name="PanelAccounts" Grid.Row="0" Visibility=
   "Collapsed">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*"/>
    </Grid.RowDefinitions>

    <Border Grid.Row="0" Background="White" Padding="
       20,14"
            BorderBrush="#E2E8F0" BorderThickness="
               0,0,0,1">
        <TextBlock Text="
                                                    "
                FontSize="18" FontWeight="SemiBold
                   " Foreground="#1A3A5C"/>
    </Border>

    <DataGrid x:Name="GridAccounts" Grid.Row="1"
              AutoGenerateColumns="False"
                 CanUserAddRows="False"
              AlternationCount="2"
                 GridLinesVisibility="Horizontal"
              HorizontalGridLinesBrush="#EDF2F7"
                 HeadersVisibility="Column"
              SelectionMode="Single" Background="
                 White" RowHeight="34"
              FontSize="12" BorderThickness="0">
        <DataGrid.ColumnHeaderStyle>
            <Style TargetType="DataGridColumnHeader">
                <Setter Property="Background"   Value
                   ="#F8FAFC"/>
                <Setter Property="Foreground"   Value
                   ="#4A5568"/>
                <Setter Property="FontWeight"   Value
                   ="SemiBold"/>
                <Setter Property="FontSize"     Value
                   ="11"/>
                <Setter Property="Padding"      Value
                   ="10,8"/>
```

```
                <Setter Property="BorderBrush"  Value
                  ="#E2E8F0"/>
                <Setter Property="BorderThickness"
                  Value="0,0,1,1"/>
            </Style>
        </DataGrid.ColumnHeaderStyle>
        <DataGrid.Columns>
            <DataGridTextColumn Header="      "
                      Binding="{Binding Code}" Width
              ="70"  IsReadOnly="True"/>
            <DataGridTextColumn Header="
                                  " Binding="{
              Binding Name}" Width="*"  IsReadOnly="
              True"/>
            <DataGridTextColumn Header="      "
                      Binding="{Binding Type}" Width
              ="90"  IsReadOnly="True">
                <DataGridTextColumn.ElementStyle>
                    <Style TargetType="TextBlock">
                        <Style.Triggers>
                            <Trigger Property="Text"
                              Value="          ">
                                <Setter Property="
                                  Foreground" Value=
                                  "#27AE60"/>
                                <Setter Property="
                                  FontWeight" Value=
                                  "SemiBold"/>
                            </Trigger>
                            <Trigger Property="Text"
                              Value="            ">
                                <Setter Property="
                                  Foreground" Value=
                                  "#E74C3C"/>
                                <Setter Property="
                                  FontWeight" Value=
                                  "SemiBold"/>
                            </Trigger>
                        </Style.Triggers>
                    </Style>
                </DataGridTextColumn.ElementStyle>
```

```
                </DataGridTextColumn>
            </DataGrid.Columns>
        </DataGrid>
    </Grid>

    <!--                        /

        -->
    <Grid x:Name="PanelReports" Grid.Row="0" Visibility="
       Collapsed">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="White" Padding="
           20,14"
                BorderBrush="#E2E8F0" BorderThickness="
                   0,0,0,1">
            <TextBlock Text="                    -
                                                    "
                        FontSize="18" FontWeight="SemiBold
                           " Foreground="#1A3A5C"/>
        </Border>

        <!--                                    -->
        <Border Grid.Row="1" Background="#F8FAFC" Padding
           ="20,12"
                BorderBrush="#E2E8F0" BorderThickness="
                   0,0,0,1">
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="                : "
                   VerticalAlignment="Center"
                            Margin="0,0,8,0" FontSize="12"
                                Foreground="#4A5568"/>
                <DatePicker x:Name="PickOsvFrom" Width="
                   130" Margin="0,0,16,0" FontSize="12"/>
```

```
            <TextBlock Text="  :  " VerticalAlignment
               ="Center"
                        Margin="0,0,8,0" FontSize="12"
                            Foreground="#4A5568"/>
            <DatePicker x:Name="PickOsvTo" Width="130
               " Margin="0,0,20,0" FontSize="12"/>
            <Button Content="
                                        " Style="{
               StaticResource ActionButton}"
                    Click="BtnBuildOsv_Click"/>
        </StackPanel>
    </Border>

    <!--                          -->
    <DataGrid x:Name="GridOsv" Grid.Row="2"
              AutoGenerateColumns="False"
                 CanUserAddRows="False"
              AlternationCount="2"
                 GridLinesVisibility="Horizontal"
              HorizontalGridLinesBrush="#EDF2F7"
                 HeadersVisibility="Column"
              SelectionMode="Single" Background="
                 White" RowHeight="34"
              FontSize="12" BorderThickness="0"
                 IsReadOnly="True">
        <DataGrid.ColumnHeaderStyle>
            <Style TargetType="DataGridColumnHeader">
                <Setter Property="Background"   Value
                   ="#EBF5FB"/>
                <Setter Property="Foreground"   Value
                   ="#1A3A5C"/>
                <Setter Property="FontWeight"   Value
                   ="SemiBold"/>
                <Setter Property="FontSize"     Value
                   ="11"/>
                <Setter Property="Padding"      Value
                   ="10,8"/>
                <Setter Property="BorderBrush"  Value
                   ="#D6EAF8"/>
                <Setter Property="BorderThickness"
                   Value="0,0,1,1"/>
```

```
            </Style>
        </DataGrid.ColumnHeaderStyle>
        <DataGrid.Columns>
            <DataGridTextColumn Header="        "
                      Binding="{Binding Account}"
                                        Width="60"
               />
            <DataGridTextColumn Header="
                                  " Binding="{
               Binding AccountName}"
                                 Width="200"/>
            <DataGridTextColumn Header="
                     .     "   Binding="{Binding
               OpenDebit,   StringFormat=N2}"  Width=
               "110"/>
            <DataGridTextColumn Header="
                     .     "   Binding="{Binding
               OpenCredit,  StringFormat=N2}"  Width=
               "110"/>
            <DataGridTextColumn Header="
                   "   Binding="{Binding TurnDebit,
               StringFormat=N2}"        Width="110"/>
            <DataGridTextColumn Header="
                   "   Binding="{Binding TurnCredit,
               StringFormat=N2}"        Width="110"/>
            <DataGridTextColumn Header="
                     .     "  Binding="{Binding
               CloseDebit,  StringFormat=N2}"   Width
               ="110"/>
            <DataGridTextColumn Header="
                     .     "  Binding="{Binding
               CloseCredit, StringFormat=N2}"   Width
               ="*"/>
        </DataGrid.Columns>
    </DataGrid>

    <!--                    -->
    <Border Grid.Row="3" Background="#EBF5FB" Padding
       ="20,10"
            BorderBrush="#D6EAF8" BorderThickness="
               0,1,0,0">
```

```
                    <TextBlock x:Name="TxtOsvTotals"
                               FontSize="12" FontWeight="SemiBold
                                  " Foreground="#1A3A5C"/>
                </Border>

                <!--                                  -->
                <Border Grid.Row="4" Background="White" Padding="
                   20,10"
                      BorderBrush="#E2E8F0" BorderThickness="
                         0,1,0,0">
                    <StackPanel Orientation="Horizontal"
                       HorizontalAlignment="Right">
                        <Button x:Name="BtnLoadOsv" Content="
                                                        (JSON)"
                            Style="{StaticResource ActionButton}"
                                Margin="0,0,8,0" Click="
                                   BtnLoadOsv_Click"/>
                        <Button x:Name="BtnExportOsv" Content="
                                                       (JSON)"
                           Style="{StaticResource ActionButton}"
                                Click="BtnExportOsv_Click"/>
                    </StackPanel>
                </Border>
            </Grid>

            <!--                                    (
                                                ,
                                                      ) -->
            <Border Grid.Row="1" Background="#1A3A5C"
                    Padding="16,5" Height="26">
                <TextBlock x:Name="TxtStatus" Text="              "
                           Foreground="#A8C7E8" FontSize="11"/>
            </Border>
        </Grid>
    </Grid>
</Window>
```

A.3. MainWindow.xaml.cs

```
using System;
using System.Collections.Generic;
```

```
using System.Collections.ObjectModel;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Threading;
using Microsoft.Win32;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;

namespace BuhUchet
{
    public partial class MainWindow : Window
    {
        private ObservableCollection<JournalEntry> _journal = new
           ();
        private ObservableCollection<Counterparty>
           _counterparties = new();
        private ObservableCollection<AccountPlan> _accounts = new
           ();

        private readonly DataService _dataService = new();
        private readonly AutoCompleteService _autoComplete = new
           ();
        private readonly string _dataPath = AppDataStore.
           DefaultPath;

        private bool _isDirty;
        private DateTime? _lastSavedAt;
        private DispatcherTimer? _saveDebounceTimer;

        private JournalEntry? _editingEntry;
        private Counterparty? _editingCp;
        private bool _isNewEntry;
        private bool _isNewCp;
        private int _nextId = 1;

        public MainWindow()
        {
            InitializeComponent();
```

```
        LoadData();
        BindGrids();
        SetupAutoComplete();
        NavigateTo("Journal");
        UpdateStatusBar();

        PickOsvFrom.SelectedDate = new DateTime(DateTime.
            Today.Year, 1, 1);
        PickOsvTo.SelectedDate = DateTime.Today;
    }

    private void LoadData()
    {
        var data = AppDataStore.LoadOrCreate(_dataPath);
        _journal = new ObservableCollection<JournalEntry>(
            data.Journal ?? new List<JournalEntry>());
        _counterparties = new ObservableCollection<
            Counterparty>(data.Counterparties ?? new List<
            Counterparty>());
        _accounts = new ObservableCollection<AccountPlan>(
            data.Accounts ?? new List<AccountPlan>());
        _nextId = _journal.Count > 0 ? _journal.Max(e => e.Id
            ) + 1 : 1;
        _lastSavedAt = data.SavedAt;
        _isDirty = false;
    }

    private void BindGrids()
    {
        GridJournal.ItemsSource = _journal;
        GridCounterparties.ItemsSource = _counterparties;
        GridAccounts.ItemsSource = _accounts;
        UpdateJournalCount();
    }

    private void SetupAutoComplete()
    {
        _autoComplete.Attach(TxtCounterparty, () =>
            _counterparties.Select(c => c.Name),
            ApplyCounterpartyToEntryByName);
```

```
        _autoComplete.Attach(TxtDebit, () => _accounts.Select
            (a => a.Code).Concat(_accounts.Select(a => a.Name)
            ));
        _autoComplete.Attach(TxtCredit, () => _accounts.
            Select(a => a.Code).Concat(_accounts.Select(a => a
            .Name)));
        _autoComplete.Attach(TxtCpName, () => _counterparties
            .Select(c => c.Name));
        _autoComplete.Attach(TxtDescription, () => _journal.
            Select(e => e.Description).Distinct());

        TxtCounterparty.TextChanged += (_, _) =>
            ApplyCounterpartyToEntryByName(TxtCounterparty.
            Text);
    }

    private void NavButton_Click(object sender,
        RoutedEventArgs e)
    {
        if (sender is Button btn && btn.Tag is string section
            )
            NavigateTo(section);
    }

    private void NavigateTo(string section)
    {
        PanelJournal.Visibility = section == "Journal" ?
            Visibility.Visible : Visibility.Collapsed;
        PanelCounterparties.Visibility = section == "
            Counterparties" ? Visibility.Visible : Visibility.
            Collapsed;
        PanelAccounts.Visibility = section == "Accounts" ?
            Visibility.Visible : Visibility.Collapsed;
        PanelReports.Visibility = section == "Reports" ?
            Visibility.Visible : Visibility.Collapsed;

        BtnJournal.Style = (Style)FindResource(section == "
            Journal" ? "NavButtonActive" : "NavButton");
        BtnCounterparties.Style = (Style)FindResource(section
            == "Counterparties" ? "NavButtonActive" : "
            NavButton");
```

```
        BtnAccounts.Style = (Style)FindResource(section == "
            Accounts" ? "NavButtonActive" : "NavButton");
        BtnReports.Style = (Style)FindResource(section == "
            Reports" ? "NavButtonActive" : "NavButton");
    }

    private void BtnAddEntry_Click(object sender,
        RoutedEventArgs e)
    {
        _editingEntry = new JournalEntry { Id = _nextId, Date
            = DateTime.Today };
        _isNewEntry = true;
        ShowEntryForm(_editingEntry);
    }

    private void GridJournal_SelectionChanged(object sender,
        SelectionChangedEventArgs e)
    {
        if (GridJournal.SelectedItem is not JournalEntry
            entry) return;
        _editingEntry = entry;
        _isNewEntry = false;
        ShowEntryForm(entry);
    }

    private void ShowEntryForm(JournalEntry entry)
    {
        PickDate.SelectedDate = entry.Date;
        TxtDebit.Text = entry.DebitAccount;
        TxtCredit.Text = entry.CreditAccount;
        TxtCounterparty.Text = entry.Counterparty;
        TxtCounterpartyInn.Text = entry.CounterpartyInn;
        TxtCounterpartyBank.Text = entry.
            CounterpartyBankAccount;
        TxtAmount.Text = entry.Amount > 0 ? entry.Amount.
            ToString("N2") : "";
        TxtDescription.Text = entry.Description;
        PanelEntryEdit.Visibility = Visibility.Visible;
    }
```

```
private void BtnSaveEntry_Click(object sender,
   RoutedEventArgs e)
{
    if (_editingEntry == null) return;
    if (PickDate.SelectedDate == null) { ShowStatus("
                                                ", true);
       return; }
    if (string.IsNullOrWhiteSpace(TxtDebit.Text)) {
       ShowStatus("                                    ",
        true); return; }
    if (string.IsNullOrWhiteSpace(TxtCredit.Text)) {
       ShowStatus("
       ", true); return; }
    if (!TryParseAmount(TxtAmount.Text, out decimal
       amount) || amount <= 0) { ShowStatus("
                                                    ",
       true); return; }

    _editingEntry.Date = PickDate.SelectedDate.Value;
    _editingEntry.DebitAccount = TxtDebit.Text.Trim();
    _editingEntry.CreditAccount = TxtCredit.Text.Trim();
    _editingEntry.Counterparty = TxtCounterparty.Text.
       Trim();
    _editingEntry.CounterpartyInn = TxtCounterpartyInn.
       Text.Trim();
    _editingEntry.CounterpartyBankAccount =
       TxtCounterpartyBank.Text.Trim();
    _editingEntry.Amount = amount;
    _editingEntry.Description = TxtDescription.Text.Trim
       ();

    if (_isNewEntry)
    {
        _journal.Add(_editingEntry);
        _nextId++;
        AutoRegisterCounterparty(_editingEntry.
           Counterparty);
    }

    PanelEntryEdit.Visibility = Visibility.Collapsed;
    _editingEntry = null;
```

```
        UpdateJournalCount();
        MarkDirty();
        ShowStatus("                                    ");
    }

    private void BtnDeleteEntry_Click(object sender,
       RoutedEventArgs e)
    {
        if (_editingEntry == null) return;
        if (!_isNewEntry)
        {
            var r = MessageBox.Show($"
                                #{_editingEntry.Id}?", "
                                        ", MessageBoxButton.
               YesNo, MessageBoxImage.Question);
            if (r != MessageBoxResult.Yes) return;
            _journal.Remove(_editingEntry);
            UpdateJournalCount();
            MarkDirty();
            ShowStatus("                                    ");
        }
        PanelEntryEdit.Visibility = Visibility.Collapsed;
        _editingEntry = null;
    }

    private void BtnCancelEntry_Click(object sender,
       RoutedEventArgs e)
    {
        PanelEntryEdit.Visibility = Visibility.Collapsed;
        _editingEntry = null;
        GridJournal.SelectedItem = null;
    }

    private void AutoRegisterCounterparty(string name)
    {
        if (string.IsNullOrWhiteSpace(name)) return;
        if (_counterparties.Any(c => c.Name.Equals(name,
           StringComparison.OrdinalIgnoreCase))) return;
        _counterparties.Add(new Counterparty { Name = name,
           Type = "             " });
    }
```

```
private void ApplyCounterpartyToEntryByName(string name)
{
    name = (name ?? "").Trim();
    var cp = _counterparties.FirstOrDefault(c => c.Name.
       Equals(name, StringComparison.OrdinalIgnoreCase));
    TxtCounterpartyInn.Text = cp?.Inn ?? "";
    TxtCounterpartyBank.Text = cp?.BankAccount ?? "";
}

private void TxtSearch_TextChanged(object sender,
   TextChangedEventArgs e)
    => ApplyJournalFilter();

private void JournalFilter_Changed(object sender,
   SelectionChangedEventArgs e)
    => ApplyJournalFilter();

private void ApplyJournalFilter()
{
    string q = TxtSearch.Text.Trim();
    DateTime? from = PickJournalFrom.SelectedDate?.Date;
    DateTime? to = PickJournalTo.SelectedDate?.Date;

    var filtered = _journal.Where(en =>
    {
        bool matchesText = string.IsNullOrWhiteSpace(q)
           ||
            en.Counterparty.Contains(q, StringComparison.
               OrdinalIgnoreCase) ||
            en.CounterpartyInn.Contains(q,
               StringComparison.OrdinalIgnoreCase) ||
            en.CounterpartyBankAccount.Contains(q,
               StringComparison.OrdinalIgnoreCase) ||
            en.Description.Contains(q, StringComparison.
               OrdinalIgnoreCase) ||
            en.DebitAccount.Contains(q, StringComparison.
               OrdinalIgnoreCase) ||
            en.CreditAccount.Contains(q, StringComparison
               .OrdinalIgnoreCase) ||
            en.Date.ToString("dd.MM.yyyy").Contains(q);
```

```
            bool matchesFrom = !from.HasValue || en.Date.Date
                >= from.Value;
            bool matchesTo = !to.HasValue || en.Date.Date <=
               to.Value;
            return matchesText && matchesFrom && matchesTo;
        }).ToList();

        GridJournal.ItemsSource = filtered;
        UpdateJournalCount();
    }

    private void BtnAddCounterparty_Click(object sender,
       RoutedEventArgs e)
    {
        _editingCp = new Counterparty();
        _isNewCp = true;
        ShowCpForm(_editingCp);
    }

    private void GridCounterparties_SelectionChanged(object
       sender, SelectionChangedEventArgs e)
    {
        if (GridCounterparties.SelectedItem is not
           Counterparty cp) return;
        _editingCp = cp;
        _isNewCp = false;
        ShowCpForm(cp);
    }

    private void ShowCpForm(Counterparty cp)
    {
        TxtCpName.Text = cp.Name;
        TxtCpInn.Text = cp.Inn;
        TxtCpBankAccount.Text = cp.BankAccount;
        TxtCpPersonalAccount.Text = cp.PersonalAccount;
        CbCpType.SelectedItem = CbCpType.Items.Cast<
           ComboBoxItem>().FirstOrDefault(i => i.Content?.
           ToString() == cp.Type);
        PanelCpEdit.Visibility = Visibility.Visible;
    }
```

```
private void BtnSaveCp_Click(object sender,
   RoutedEventArgs e)
{
    if (_editingCp == null) return;
    if (string.IsNullOrWhiteSpace(TxtCpName.Text)) {
       ShowStatus("
                                                        ",
       true); return; }

    bool isExisting = !_isNewCp;
    string oldName = _editingCp.Name;

    _editingCp.Name = TxtCpName.Text.Trim();
    _editingCp.Inn = TxtCpInn.Text.Trim();
    _editingCp.BankAccount = TxtCpBankAccount.Text.Trim()
       ;
    _editingCp.PersonalAccount = TxtCpPersonalAccount.
       Text.Trim();
    _editingCp.Type = (CbCpType.SelectedItem as
       ComboBoxItem)?.Content?.ToString() ?? "
                   ";

    if (_isNewCp) _counterparties.Add(_editingCp);

    //

                                                        ,
    //                  /         .

                                 data.json.
    if (isExisting)
        SyncJournalEntriesForCounterparty(oldName,
           _editingCp);

    PanelCpEdit.Visibility = Visibility.Collapsed;
    _editingCp = null;
    MarkDirty();
    ShowStatus("                                        ");
}
```

```
private void SyncJournalEntriesForCounterparty(string
    oldName, Counterparty cp)
{
    string newName = (cp.Name ?? "").Trim();
    if (string.IsNullOrWhiteSpace(oldName) && string.
        IsNullOrWhiteSpace(newName)) return;

    foreach (var entry in _journal)
    {
        bool nameMatchesOld = !string.IsNullOrWhiteSpace(
            oldName) &&
                                entry.Counterparty.Equals(
                                    oldName,
                                    StringComparison.
                                    OrdinalIgnoreCase);
        bool nameMatchesNew = !string.IsNullOrWhiteSpace(
            newName) &&
                                entry.Counterparty.Equals(
                                    newName,
                                    StringComparison.
                                    OrdinalIgnoreCase);

        if (!nameMatchesOld && !nameMatchesNew) continue;

        //

        if (nameMatchesOld && !string.IsNullOrWhiteSpace(
            newName))
            entry.Counterparty = newName;

        entry.CounterpartyInn = cp.Inn ?? "";
        entry.CounterpartyBankAccount = cp.BankAccount ??
            "";
    }

    //


    ApplyCounterpartyToEntryByName(TxtCounterparty.Text);
    ApplyJournalFilter();
}
```

```
private void BtnJournalReset_Click(object sender,
   RoutedEventArgs e)
{
    PickJournalFrom.SelectedDate = null;
    PickJournalTo.SelectedDate = null;
    ApplyJournalFilter();
}

private void BtnDeleteCp_Click(object sender,
   RoutedEventArgs e)
{
    if (_editingCp == null || _isNewCp) return;
    var r = MessageBox.Show($"                    {
       _editingCp.Name}  ?", "                          "
       , MessageBoxButton.YesNo, MessageBoxImage.Question
       );
    if (r != MessageBoxResult.Yes) return;
    _counterparties.Remove(_editingCp);
    PanelCpEdit.Visibility = Visibility.Collapsed;
    _editingCp = null;
    MarkDirty();
    ShowStatus("                                  ");
}

private void BtnBuildOsv_Click(object sender,
   RoutedEventArgs e)
{
    if (PickOsvFrom.SelectedDate == null || PickOsvTo.
       SelectedDate == null)
    {
        ShowStatus("                              ", true);
        return;
    }

    var from = PickOsvFrom.SelectedDate.Value;
    var to = PickOsvTo.SelectedDate.Value;
    var rows = _dataService.BuildOsv(_journal.ToList(),
       _accounts.ToList(), from, to);
    GridOsv.ItemsSource = rows;
```

```
        TxtOsvTotals.Text =
            $"          :                    = {rows.Sum(r =>
               r.TurnDebit):N2}  |  " +
            $"                  = {rows.Sum(r => r.TurnCredit
               ):N2}  |  " +
            $"                   .      = {rows.Sum(r => r.
               CloseDebit):N2}  |  " +
            $"                   .      = {rows.Sum(r => r.
               CloseCredit):N2}";

        ShowStatus($"                                : {rows.
           Count}             ");
    }

    private void BtnExportOsv_Click(object sender,
       RoutedEventArgs e)
    {
        if (PickOsvFrom.SelectedDate == null || PickOsvTo.
           SelectedDate == null)
        {
            ShowStatus("
                                ", true);
            return;
        }

        var from = PickOsvFrom.SelectedDate.Value;
        var to = PickOsvTo.SelectedDate.Value;
        var rows = (GridOsv.ItemsSource as IEnumerable<OsvRow
           >)?.ToList() ??
                _dataService.BuildOsv(_journal.ToList(),
                   _accounts.ToList(), from, to);
        var usedCounterparties = _journal
            .Where(j => j.Date >= from && j.Date <= to && !
               string.IsNullOrWhiteSpace(j.Counterparty))
            .Select(j => j.Counterparty.Trim())
            .Distinct(StringComparer.OrdinalIgnoreCase)
            .Select(name =>
            {
                var cp = _counterparties.FirstOrDefault(c =>
                   c.Name.Equals(name, StringComparison.
                   OrdinalIgnoreCase));
```

```
            return new
            {
                Name = name,
                Inn = cp?.Inn ?? "",
                PersonalAccount = cp?.PersonalAccount ??
                    ""
            };
        })
        .ToList();

    var reportData = new
    {
        Report = "      ",
        PeriodFrom = from.ToString("yyyy-MM-dd"),
        PeriodTo = to.ToString("yyyy-MM-dd"),
        Counterparties = usedCounterparties,
        Rows = rows,
        Totals = new
        {
            TurnDebit = rows.Sum(r => r.TurnDebit),
            TurnCredit = rows.Sum(r => r.TurnCredit),
            CloseDebit = rows.Sum(r => r.CloseDebit),
            CloseCredit = rows.Sum(r => r.CloseCredit)
        }
    };

    var dlg = new SaveFileDialog
    {
        Filter = "JSON-         (*.json)|*.json",
        Title = "                      ",
        DefaultExt = "json",
        FileName = $"   _   {from:yyyyMMdd}_{to:yyyyMMdd}
            "
    };
    if (dlg.ShowDialog() != true) return;

    try
    {
        File.WriteAllText(dlg.FileName, JsonConvert.
            SerializeObject(reportData, Formatting.
            Indented));
```

```
            ShowStatus($"                                : {Path.
               GetFileName(dlg.FileName)}");
        }
        catch (Exception ex)
        {
            ShowStatus($"                                : {ex.
               Message}", true);
        }
    }

    private void BtnLoadOsv_Click(object sender,
       RoutedEventArgs e)
    {
        var dlg = new OpenFileDialog
        {
            Filter = "JSON-          (*.json)|*.json",
            Title = "                                    "
        };
        if (dlg.ShowDialog() != true) return;

        try
        {
            string json = File.ReadAllText(dlg.FileName);
            var root = JObject.Parse(json);

            var rows = root["Rows"]?.ToObject<List<OsvRow>>()
                ?? new List<OsvRow>();
            GridOsv.ItemsSource = rows;

            DateTime parsedFrom;
            DateTime parsedTo;
            if (DateTime.TryParse(root["PeriodFrom"]?.
               ToString(), out parsedFrom))
                PickOsvFrom.SelectedDate = parsedFrom;
            if (DateTime.TryParse(root["PeriodTo"]?.ToString
               (), out parsedTo))
                PickOsvTo.SelectedDate = parsedTo;

            var totals = root["Totals"];
            decimal sumTurnDt = totals?["TurnDebit"]?.Value<
               decimal>() ?? rows.Sum(r => r.TurnDebit);
```

```
            decimal sumTurnCt = totals?["TurnCredit"]?.Value<
               decimal>() ?? rows.Sum(r => r.TurnCredit);
            decimal sumCloseDt = totals?["CloseDebit"]?.Value
               <decimal>() ?? rows.Sum(r => r.CloseDebit);
            decimal sumCloseCt = totals?["CloseCredit"]?.
               Value<decimal>() ?? rows.Sum(r => r.
               CloseCredit);

            TxtOsvTotals.Text =
                $"          :                    = {sumTurnDt:
                   N2}  |  " +
                $"                  = {sumTurnCt:N2}  |  " +
                $"                   .      = {sumCloseDt:N2}
                     |  " +
                $"                   .      = {sumCloseCt:N2}
                   ";

            ShowStatus($"                            : {Path.
               GetFileName(dlg.FileName)}");
        }
        catch (Exception ex)
        {
            ShowStatus($"
                           : {ex.Message}", true);
        }
    }

    private void MarkDirty()
    {
        _isDirty = true;
        UpdateStatusBar();
        ScheduleSave();
    }

    private void ScheduleSave()
    {
        _saveDebounceTimer ??= new DispatcherTimer { Interval
            = TimeSpan.FromMilliseconds(600) };
        _saveDebounceTimer.Stop();
        _saveDebounceTimer.Tick -= SaveDebounceTimer_Tick;
        _saveDebounceTimer.Tick += SaveDebounceTimer_Tick;
```

```
        _saveDebounceTimer.Start();
    }

    private void SaveDebounceTimer_Tick(object? sender,
       EventArgs e)
    {
        _saveDebounceTimer?.Stop();
        TrySaveData();
    }

    private void TrySaveData()
    {
        if (!_isDirty) return;
        try
        {
            var data = new SaveFile
            {
                Journal = _journal.ToList(),
                Counterparties = _counterparties.ToList(),
                Accounts = _accounts.ToList(),
                SavedAt = DateTime.Now
            };
            AppDataStore.Save(_dataPath, data);
            _isDirty = false;
            _lastSavedAt = data.SavedAt;
            UpdateStatusBar();
        }
        catch (Exception ex)
        {
            ShowStatus($"
                                        : {ex.Message}",
               true);
        }
    }

    private void UpdateStatusBar()
    {
        TxtFileName.Text = "data.json";
        TxtSavedAt.Text = _isDirty
            ? "
                                (
```

```
                                        )"
        : _lastSavedAt.HasValue ? $"                    : {
           _lastSavedAt:dd.MM.yyyy HH:mm}" : "
                                ";
}

private void UpdateJournalCount()
{
    int total = _journal.Count;
    decimal sum = _journal.Sum(e => e.Amount);
    TxtJournalCount.Text = $"{total}                    |
                : {sum:N2}        .";
}

private void ShowStatus(string message, bool isError =
   false)
{
    TxtStatus.Text = isError ? $"    {message}" : $"    {
       message}";
    TxtStatus.Foreground = isError ? System.Windows.Media
       .Brushes.LightSalmon : System.Windows.Media.
       Brushes.LightGreen;
}

private static bool TryParseAmount(string text, out
   decimal amount)
{
    text = (text ?? "").Trim().Replace(" ", "");
    if (decimal.TryParse(text, NumberStyles.Any,
       CultureInfo.GetCultureInfo("ru-RU"), out amount))
       return true;
    return decimal.TryParse(text.Replace(",", "."),
       NumberStyles.Any, CultureInfo.InvariantCulture,
       out amount);
}

protected override void OnClosing(System.ComponentModel.
   CancelEventArgs e)
{
    TrySaveData();
    base.OnClosing(e);
```

```
        }
    }
}
```

A.4. Models.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace BuhUchet
{
    public abstract class NotifyObject : INotifyPropertyChanged
    {
        public event PropertyChangedEventHandler? PropertyChanged
            ;

        protected void OnPropertyChanged([CallerMemberName]
            string? name = null)
             => PropertyChanged?.Invoke(this, new
                PropertyChangedEventArgs(name));
    }

    //                    (                                    )
    public class JournalEntry : NotifyObject
    {
        private int _id;
        private DateTime _date;
        private string _debitAccount  = "";
        private string _creditAccount = "";
        private string _counterparty  = "";
        private string _counterpartyInn = "";
        private string _counterpartyBankAccount = "";
        private string _description   = "";
        private decimal _amount;

        public int       Id              { get => _id;
            set { _id = value;              OnPropertyChanged(); }
            }
```

```
    public DateTime Date            { get => _date;
       set { _date = value;            OnPropertyChanged(); }
       }
    public string   DebitAccount    { get => _debitAccount;
       set { _debitAccount = value;    OnPropertyChanged(); }
       }
    public string   CreditAccount   { get => _creditAccount;
       set { _creditAccount = value;   OnPropertyChanged(); }
       }
    public string   Counterparty    { get => _counterparty;
       set { _counterparty = value;    OnPropertyChanged(); }
       }
    public string   CounterpartyInn { get => _counterpartyInn
       ; set { _counterpartyInn = value; OnPropertyChanged();
        } }
    public string   CounterpartyBankAccount { get =>
       _counterpartyBankAccount; set {
       _counterpartyBankAccount = value; OnPropertyChanged();
        } }
    public string   Description     { get => _description;
       set { _description = value;     OnPropertyChanged(); }
       }
    public decimal  Amount          { get => _amount;
       set { _amount = value;          OnPropertyChanged(); }
       }
}

//
public class Counterparty : NotifyObject
{
    private string _name = "";
    private string _inn = "";
    private string _bankAccount = "";
    private string _personalAccount = "";
    private string _type = "";

    public string Name              { get => _name;
       set { _name = value;             OnPropertyChanged(); }
        }
    public string Inn               { get => _inn;
       set { _inn = value;              OnPropertyChanged(); }
```

```
        }
    public string BankAccount     { get => _bankAccount;
       set { _bankAccount = value;     OnPropertyChanged(); }
        } //
    public string PersonalAccount { get => _personalAccount;
       set { _personalAccount = value; OnPropertyChanged(); }
        } //
    public string Type            { get => _type;
       set { _type = value;            OnPropertyChanged(); }
        } //                    /                      /

}

//
public class AccountPlan : NotifyObject
{
    private string _code = "";
    private string _name = "";
    private string _type = "";

    public string Code        { get => _code; set { _code =
       value; OnPropertyChanged(); } }
    public string Name        { get => _name; set { _name =
       value; OnPropertyChanged(); } }
    public string Type        { get => _type; set { _type =
       value; OnPropertyChanged(); } } //            /
                    /
}

//                      (                -
                     )
public class OsvRow
{
    public string  Account     { get; set; } = "";
    public string  AccountName { get; set; } = "";
    public decimal OpenDebit   { get; set; }
    public decimal OpenCredit  { get; set; }
    public decimal TurnDebit   { get; set; }
    public decimal TurnCredit  { get; set; }
    public decimal CloseDebit  => Math.Max(0, OpenDebit  -
       OpenCredit  + TurnDebit  - TurnCredit);
```

```
        public decimal CloseCredit => Math.Max(0, OpenCredit -
            OpenDebit   + TurnCredit - TurnDebit);
    }

    //
    public class SaveFile
    {
        public List<JournalEntry> Journal       { get; set; } =
            new();
        public List<Counterparty> Counterparties { get; set; } =
            new();
        public List<AccountPlan>  Accounts       { get; set; } =
            new();
        public DateTime           SavedAt        { get; set; } =
            DateTime.Now;
    }
}
```

A.5. DataService.cs

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace BuhUchet
{
    /// <summary>
    ///                                        .
    /// </summary>
    public class DataService
    {
        //        (                -
                                )
        public List<OsvRow> BuildOsv(
            List<JournalEntry> journal,
            List<AccountPlan> accounts,
            DateTime from, DateTime to)
        {
            //

            var allCodes = journal
```

```
            .SelectMany(e => new[] { e.DebitAccount, e.
               CreditAccount })
            .Distinct()
            .OrderBy(c => c)
            .ToList();

        var rows = new List<OsvRow>();

        foreach (var code in allCodes)
        {
            var name = accounts.FirstOrDefault(a => a.Code ==
                code)?.Name ?? "    ";

            //
            decimal turnDt = journal
                .Where(e => e.Date >= from && e.Date <= to &&
                    e.DebitAccount == code)
                .Sum(e => e.Amount);
            decimal turnCt = journal
                .Where(e => e.Date >= from && e.Date <= to &&
                    e.CreditAccount == code)
                .Sum(e => e.Amount);

            //                                    (
                                )
            decimal preDt = journal
                .Where(e => e.Date < from && e.DebitAccount
                   == code)
                .Sum(e => e.Amount);
            decimal preCt = journal
                .Where(e => e.Date < from && e.CreditAccount
                   == code)
                .Sum(e => e.Amount);

            rows.Add(new OsvRow
            {
                Account     = code,
                AccountName = name,
                OpenDebit   = preDt,
                OpenCredit  = preCt,
                TurnDebit   = turnDt,
```

```
                    TurnCredit  = turnCt
                });
            }

            return rows;
        }

        //
        public List<JournalEntry> FilterByCounterparty(
            List<JournalEntry> journal, string counterparty)
            => journal
                .Where(e => e.Counterparty.Equals(counterparty,
                   StringComparison.OrdinalIgnoreCase))
                .OrderBy(e => e.Date)
                .ToList();

    }
}
```

A.6. AutoCompleteService.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;

namespace BuhUchet
{
    /// <summary>
    ///                                                    :
                        Tab
                            .
    ///                                    TextBox
                          Attach().
    /// </summary>
    public class AutoCompleteService
    {
        private readonly Dictionary<TextBox, AutoCompleteState>
           _states = new();
```

```
//                          TextBox
public void Attach(TextBox textBox, Func<IEnumerable<
   string>> sourceProvider, Action<string>? onSelected =
   null)
{
    var state = new AutoCompleteState(sourceProvider,
       onSelected);
    _states[textBox] = state;

    textBox.PreviewKeyDown += (s, e) =>
    {
        if (e.Key == Key.Tab)
        {
            e.Handled = HandleTab(textBox, state);
        }
        else if (e.Key == Key.Escape)
        {
            HidePopup(state);
        }
        else if (e.Key == Key.Down && state.Popup?.IsOpen
            == true)
        {
            if (state.ListBox != null && state.ListBox.
               Items.Count > 0)
            {
                state.ListBox.Focus();
                state.ListBox.SelectedIndex = 0;
            }
            e.Handled = true;
        }
    };

    textBox.TextChanged += (s, e) =>
    {
        //                                          2+

        if (textBox.Text.Length >= 2)
            ShowSuggestions(textBox, state);
        else
            HidePopup(state);
```

```
        };

        textBox.LostFocus += (s, e) =>
        {
            //
                                ,

            textBox.Dispatcher.BeginInvoke(System.Windows.
               Threading.DispatcherPriority.Background,
                () => HidePopup(state));
        };
    }

    //                    Tab
    private bool HandleTab(TextBox textBox, AutoCompleteState
        state)
    {
        string prefix = textBox.Text.Trim();
        if (string.IsNullOrWhiteSpace(prefix)) return false;

        var matches = GetMatches(state, prefix).ToList();

        if (matches.Count == 1)
        {
            //

            textBox.Text = matches[0];
            textBox.CaretIndex = textBox.Text.Length;
            HidePopup(state);
            state.OnSelected?.Invoke(matches[0]);
            return true;
        }
        else if (matches.Count > 1)
        {
            ShowSuggestions(textBox, state);
            return true;
        }

        return false;
    }
```

```
//                    popup

private void ShowSuggestions(TextBox textBox,
   AutoCompleteState state)
{
    var matches = GetMatches(state, textBox.Text.Trim()).
       ToList();
    if (matches.Count == 0) { HidePopup(state); return; }

    if (state.Popup == null)
        BuildPopup(textBox, state);

    state.ListBox!.ItemsSource = matches;
    state.Popup!.IsOpen = true;
}

private void HidePopup(AutoCompleteState state)
{
    if (state.Popup != null)
        state.Popup.IsOpen = false;
}

//                  Popup + ListBox
private void BuildPopup(TextBox textBox,
   AutoCompleteState state)
{
    var listBox = new ListBox
    {
        MaxHeight = 160,
        FontSize = 13,
        Background = System.Windows.Media.Brushes.White,
        BorderBrush = System.Windows.Media.Brushes.
           LightGray,
        BorderThickness = new Thickness(1)
    };

    listBox.MouseLeftButtonUp += (s, e) =>
    {
        if (listBox.SelectedItem is string selected)
        {
            textBox.Text = selected;
```

```
            textBox.CaretIndex = textBox.Text.Length;
            HidePopup(state);
            state.OnSelected?.Invoke(selected);
            textBox.Focus();
        }
    };

    listBox.KeyDown += (s, e) =>
    {
        if (e.Key == Key.Enter && listBox.SelectedItem is
            string selected)
        {
            textBox.Text = selected;
            textBox.CaretIndex = textBox.Text.Length;
            HidePopup(state);
            state.OnSelected?.Invoke(selected);
            textBox.Focus();
            e.Handled = true;
        }
        else if (e.Key == Key.Escape)
        {
            HidePopup(state);
            textBox.Focus();
            e.Handled = true;
        }
    };

    var popup = new System.Windows.Controls.Primitives.
       Popup
    {
        PlacementTarget = textBox,
        Placement = System.Windows.Controls.Primitives.
           PlacementMode.Bottom,
        StaysOpen = false,
        AllowsTransparency = true,
        Child = listBox,
        Width = Math.Max(textBox.ActualWidth, 200)
    };

    state.Popup = popup;
    state.ListBox = listBox;
```

```
        }

        private IEnumerable<string> GetMatches(AutoCompleteState
            state, string prefix)
        {
            if (string.IsNullOrWhiteSpace(prefix)) return
                Enumerable.Empty<string>();
            return state.SourceProvider()
                .Where(s => s.StartsWith(prefix, StringComparison
                    .OrdinalIgnoreCase))
                .OrderBy(s => s)
                .Take(10);
        }
    }

    //                                          TextBox
    internal class AutoCompleteState
    {
        public Func<IEnumerable<string>> SourceProvider { get; }
        public Action<string>? OnSelected { get; }
        public System.Windows.Controls.Primitives.Popup? Popup {
            get; set; }
        public ListBox? ListBox { get; set; }

        public AutoCompleteState(Func<IEnumerable<string>>
            sourceProvider, Action<string>? onSelected)
        {
            SourceProvider = sourceProvider;
            OnSelected = onSelected;
        }
    }
}
```

A.7. AppDataStore.cs

```
using System;
using System.IO;
using Newtonsoft.Json;

namespace BuhUchet
{
```

```
internal static class AppDataStore
{
    private static readonly JsonSerializerSettings
       JsonSettings = new()
    {
        Formatting = Formatting.Indented,
        DateFormatString = "yyyy-MM-dd"
    };

    public static string DefaultPath =>
        Path.Combine(AppContext.BaseDirectory, "data.json");

    public static SaveFile LoadOrCreate(string path)
    {
        try
        {
            if (!File.Exists(path))
            {
                var initial = new SaveFile
                {
                    Journal = new(),
                    Counterparties = new(),
                    Accounts = new(),
                    SavedAt = DateTime.Now
                };
                Save(path, initial);
                return initial;
            }

            string json = File.ReadAllText(path);
            return JsonConvert.DeserializeObject<SaveFile>(
               json, JsonSettings) ?? new SaveFile();
        }
        catch
        {
            var fallback = new SaveFile
            {
                Journal = new(),
                Counterparties = new(),
                Accounts = new(),
                SavedAt = DateTime.Now
```

```
                };
                Save(path, fallback);
                return fallback;
            }
        }

        public static void Save(string path, SaveFile data)
        {
            data.SavedAt = DateTime.Now;
            string json = JsonConvert.SerializeObject(data,
               JsonSettings);
            File.WriteAllText(path, json);
        }
    }
}
```